

DEEP LEARNING LABORATORY

End Semester Practical File

Course Code: AI 524

Object Detection • Face Mask Detection • CNN Feature Maps

Name	Raghav Sharma
Roll No.	25201313
Course Code	AI 524
Subject	Deep Learning Laboratory
Assessment	End Sem Practical

Index

Ex. No.	Title	Domain
1	Real-time object detection & tracking in video feeds using YOLOv8 and SSD on the COCO dataset	Security / Vision AI
2	Real-time face mask detection using transfer learning with MobileNetV2	Health / Surveillance
3	Build a CNN (Conv → ReLU → MaxPool) and visualize feature maps on MNIST	Model Interpretability

Experiment 1: Object Detection & Tracking in Video Feeds (YOLO & SSD)

Aim: To implement a real-time object detection and tracking system for video feeds using YOLO and SSD models trained on the COCO dataset, for use in intelligent monitoring / security applications.

Domain: Security, Vision AI.

Models: YOLOv8 (single-stage anchor-free detector) and SSD300-VGG16 (single-shot multibox detector).

Dataset: COCO (80 object classes); the COCO128 subset is used for fine-tuning demonstration.

Application: Intelligent monitoring systems that detect, label, track and count objects across video frames.

Theory

Object detection answers two questions for every object in an image: what is it (classification) and where is it (a bounding box). Modern detectors fall into two broad families. Single-stage detectors such as YOLO (You Only Look Once) and SSD (Single Shot MultiBox Detector) predict class scores and box coordinates in one forward pass over a dense grid of locations, which makes them fast enough for real-time video. Two-stage detectors first propose candidate regions and then classify them, trading speed for a little extra accuracy.

YOLOv8 is an anchor-free, single-stage network. Its backbone (a CSP-style convolutional network with C2f blocks and an SPPF layer) extracts multi-scale features; a neck (a feature pyramid with up-sampling and concatenation) fuses these scales; and a decoupled detection head predicts boxes and classes at three resolutions. Non-maximum suppression (NMS) removes overlapping duplicate boxes. The model is pretrained on COCO's 80 classes, so it can be used directly or fine-tuned on a custom set.

SSD300-VGG16 uses a VGG16 backbone and predicts boxes from feature maps at several scales using predefined default (anchor) boxes. It is a useful baseline to compare against YOLO in terms of speed and detection quality.

Tracking extends detection across time by assigning a persistent ID to each object so the same object keeps its identity from frame to frame. ByteTrack (used here) associates detections between frames using motion and overlap (IoU), keeping even low-confidence boxes to recover objects that briefly fade. By collecting the set of unique IDs seen per class, the system can count how many distinct objects passed through the scene.

Procedure

- Mount Google Drive and install the Ultralytics package, which provides YOLOv8 and the built-in ByteTrack / BoT-SORT trackers.
- Load the pretrained yolov8n.pt (nano) weights and run inference on a sample image to verify the 80 COCO class labels.
- Write a coco128.yaml data configuration pointing at the Drive dataset paths and fine-tune the model for 20 epochs.
- Run tracking on a test video with ByteTrack and persist=True so each detection keeps a stable track ID.
- Loop over frames, draw boxes with their track IDs, and maintain a per-class set of unique IDs to count distinct objects.
- Repeat detection with SSD300-VGG16 from torchvision and compare the output.

Source Code

1. Setup and YOLOv8 inference

```
from google.colab import drive
drive.mount('/content/drive')

# Ultralytics ships YOLOv8/v11 + built-in trackers (ByteTrack, BoT-SORT)
!pip install -q ultralytics
import ultralytics
ultralytics.checks()                # prints GPU / versions

from ultralytics import YOLO
# 'n' = nano (fastest, best for "real-time"); use s/m/l/x for more accuracy
model = YOLO("yolov8n.pt")          # already trained on COCO's 80 classes

# Quick test on one of your dataset images
import os
img_dir = "/content/drive/MyDrive/Datasets/coco128/images/train2017"
sample = os.path.join(img_dir, sorted(os.listdir(img_dir))[0])
results = model(sample)
results[0].show()                   # renders the annotated image inline
print(model.names)                  # the 80 COCO class labels
```

2. Fine-tuning on COCO128

```
# Write a dataset config that points at YOUR drive paths
yaml_text = """
path: /content/drive/MyDrive/Datasets/coco128
train: images/train2017
val:   images/train2017          # reusing train as val (demo only)
names:                                # COCO's 80 class names
  0: person
  1: bicycle
  2: car
  ...
  79: toothbrush
"""

with open("coco128.yaml", "w") as f:
    f.write(yaml_text)

# Fine-tune (start from pretrained weights)
model = YOLO("yolov8n.pt")
model.train(data="coco128.yaml", epochs=20, imgsz=640, batch=16)

# Best weights are saved here:
best = "runs/detect/train/weights/best.pt"
print("Trained weights:", best)
```

3. Object tracking on a video with ByteTrack

```
# Download a ready-made test video directly (no upload needed)
!wget -q https://github.com/intel-iot-devkit/sample-videos/raw/master/person-bicycle-car-detection.mp4
-O test.mp4

video_path = "test.mp4"
model = YOLO("yolov8n.pt")
results = model.track(
    source=video_path,
    tracker="bytetrack.yaml",
    persist=True,
    save=True,
    conf=0.4
)
print("Annotated video saved under runs/detect/track/")
```

4. Per-class unique object counting

```
import cv2
from collections import defaultdict

model = YOLO("yolov8n.pt")
cap = cv2.VideoCapture(video_path)
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS) or 30
out = cv2.VideoWriter("monitored.mp4",
                     cv2.VideoWriter_fourcc(*"mp4v"), fps, (w, h))

seen_ids = defaultdict(set)      # class_name -> {unique track ids ever seen}

while cap.isOpened():
    ok, frame = cap.read()
    if not ok:
        break
    res = model.track(frame, persist=True, tracker="bytetrack.yaml",
                      conf=0.4, verbose=False)[0]
    if res.boxes.id is not None:
        ids = res.boxes.id.int().cpu().tolist()
        cls = res.boxes.cls.int().cpu().tolist()
        boxes = res.boxes.xyxy.cpu().numpy()
        for tid, c, (x1, y1, x2, y2) in zip(ids, cls, boxes):
            name = model.names[c]
            seen_ids[name].add(tid)
            cv2.rectangle(frame, (int(x1), int(y1)), (int(x2), int(y2)),
                          (0, 255, 0), 2)
            cv2.putText(frame, f"{name} #{tid}", (int(x1), int(y1) - 8),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
    # overlay totals (unique objects counted so far)
    y = 30
    for name, s in sorted(seen_ids.items()):
        cv2.putText(frame, f"{name}: {len(s)}", (10, y),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
        y += 28
    out.write(frame)

cap.release(); out.release()
print("Saved monitored.mp4")
```

5. SSD300-VGG16 detection (comparison)

```
import torch, torchvision
from torchvision.models.detection import ssd300_vgg16, SSD300_VGG16_Weights
from torchvision.utils import draw_bounding_boxes
from torchvision.io import read_image
import torchvision.transforms.functional as F

weights = SSD300_VGG16_Weights.DEFAULT
ssd = ssd300_vgg16(weights=weights).eval()
labels = weights.meta["categories"]
preprocess = weights.transforms()

img = read_image(sample) # the sample from the YOLO cell
with torch.no_grad():
    pred = ssd([preprocess(img))][0]

keep = pred["scores"] > 0.4
boxes = pred["boxes"][keep]
names = [f"{labels[i]} {s:.2f}"
         for i, s in zip(pred["labels"][keep], pred["scores"][keep])]

out = draw_bounding_boxes(img, boxes, names, width=3, colors="lime")
F.to_pil_image(out)
```

Output

YOLOv8 detection on a sample COCO image. Each detected object is enclosed in a coloured box with its class label and confidence score (for example, the bowls, broccoli and hot dog below).

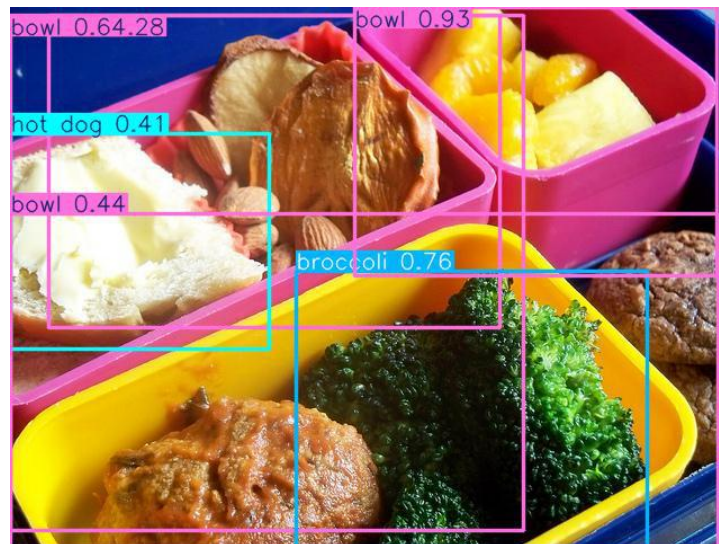


Figure 1.1 — YOLOv8n detections with class labels and confidence scores.

Fine-tuning ran for 20 epochs on COCO128. Validation metrics improved steadily, reaching the values shown below on the final epoch.

```
20 epochs completed in 0.023 hours.
Validating runs/detect/train/weights/best.pt ...
Class      Images  Instances  Box(P  R      mAP50  mAP50-95)
all         128      929        0.724  0.688  0.764  0.581
person      61       254        0.845  0.685  0.791  0.579
car         12       46         0.748  0.258  0.400  0.227
```

Speed: 0.2ms preprocess, 2.5ms inference, 8.3ms postprocess per image

During video tracking, ByteTrack kept a stable identity for each object across frames (e.g. one person tracked continuously through frames 16–349, then a person + bicycle, then a car). The annotated video and a counted version (monitored.mp4) were saved under runs/detect/track/.

```
video 1/1 (frame 327/647) test.mp4: 384x640 1 person, 1 bicycle, 5.9ms
video 1/1 (frame 567/647) test.mp4: 384x640 1 person, 1 bicycle, 1 car, 6.4ms
Speed: 2.3ms preprocess, 6.8ms inference per image at shape (1, 3, 384, 640)
Results saved to runs/detect/track
Saved monitored.mp4
```

SSD300-VGG16 was run on the same images for comparison. It detected a person in a live webcam capture and the containers in the sample image, drawing boxes in lime green.



Figure 1.2 — SSD300-VGG16 detections: a person (left) and food containers (right).

Result

Real-time object detection and tracking was implemented successfully. The fine-tuned YOLOv8n model achieved an overall mAP50 of 0.764 and mAP50-95 of 0.581 on COCO128, running at roughly 2.5 ms inference per image. ByteTrack maintained persistent IDs across frames, enabling unique object counting per class, and SSD300-VGG16 produced comparable detections — fulfilling the objective of an intelligent monitoring pipeline.

Experiment 2: Real-time Face Mask Detection using Transfer Learning (MobileNetV2)

Aim: To develop a real-time face mask detection system using transfer learning with a MobileNetV2 backbone, classifying faces as With Mask or Without Mask.

Model: MobileNetV2 (ImageNet-pretrained) as a frozen feature extractor with a custom classification head.

Dataset: Face Mask Dataset (Kaggle) — images with PASCAL-VOC XML annotations.

Expected output: Bounding boxes around faces with a real-time prediction label (Mask / No Mask) and confidence.

Theory

Transfer learning reuses a network already trained on a large dataset (ImageNet) and adapts it to a new, smaller task. The early convolutional layers have already learned general visual features (edges, textures, shapes), so only a small new head needs to be trained. This gives strong accuracy with far less data and compute than training from scratch.

MobileNetV2 is a lightweight architecture designed for mobile and real-time use. It is built from inverted residual blocks with depthwise-separable convolutions: a depthwise convolution filters each channel independently and a pointwise (1x1) convolution mixes channels, drastically reducing the number of parameters and multiply-add operations compared with standard convolutions. Linear bottlenecks and residual shortcuts preserve information through the network.

In this experiment the pretrained MobileNetV2 backbone is frozen (its weights are not updated). A new head is attached: average pooling, flattening, a 128-unit ReLU dense layer, dropout (0.5) for regularisation, and a 2-unit softmax that outputs the With Mask / Without Mask probabilities. Only this head (about 164k parameters) is trained.

For inference on full images, a separate OpenCV DNN face detector (a ResNet-10 SSD Caffe model) first locates faces; each detected face is cropped, preprocessed and passed to the trained mask classifier, and the result is drawn as a coloured box and label.

The model is judged with three metrics. Accuracy is the fraction of all predictions that are correct. Precision is the fraction of predicted positives that are truly positive. Recall is the fraction of actual positives that were detected. For mask detection, high recall on the With Mask class matters because missed masks undermine the safety purpose of the system.

Procedure

- Parse every XML annotation, crop each annotated face, convert to RGB, resize to 224x224, and label it With Mask (1) or Without Mask (0).
- Split the data 80/20 (stratified) and build tf.data pipelines that apply MobileNetV2 preprocessing per batch.
- Load MobileNetV2 without its top, freeze it, and attach the custom classification head.
- Compile with Adam (learning rate 1e-4) and categorical cross-entropy, then train for 10 epochs.
- Evaluate accuracy, precision and recall, and print a full classification report; save the model.
- Run inference: detect faces with the OpenCV DNN detector, classify each face, and overlay the label and box.

Source Code

1. Load and preprocess the dataset

```
import os, gc, cv2
import numpy as np
import xml.etree.ElementTree as ET
from tqdm import tqdm
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from tensorflow.keras.layers import AveragePooling2D, Dropout, Flatten, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
from sklearn.metrics import (classification_report, accuracy_score,
                             precision_score, recall_score)

DATASET_DIR = "/content/drive/MyDrive/Datasets/FaceMaskDetection"
ANNOTATION_DIR = os.path.join(DATASET_DIR, "annotations")
IMAGE_DIR = os.path.join(DATASET_DIR, "images")
IMG_SIZE = 224

data, labels = [], []
for xml_file in tqdm(os.listdir(ANNOTATION_DIR)):
    tree = ET.parse(os.path.join(ANNOTATION_DIR, xml_file))
    root = tree.getroot()
    image_path = os.path.join(IMAGE_DIR, root.find("filename").text)
    if not os.path.exists(image_path):
        continue
    image = cv2.imread(image_path)
    if image is None:
        continue
    for obj in root.findall("object"):
        label = obj.find("name").text
        b = obj.find("bndbox")
        xmin, ymin = int(b.find("xmin").text), int(b.find("ymin").text)
        xmax, ymax = int(b.find("xmax").text), int(b.find("ymax").text)
        face = image[ymin:ymax, xmin:xmax]
        if face.size == 0:
            continue
        face = cv2.cvtColor(face, cv2.COLOR_BGR2RGB)
        face = cv2.resize(face, (IMG_SIZE, IMG_SIZE))
        data.append(face.astype("uint8")) # raw uint8, no preprocessing yet
        labels.append(1 if label == "with_mask" else 0)

data = np.array(data, dtype="uint8") # ~0.6 GB instead of ~2.3 GB
```

```

labels = to_categorical(np.array(labels), num_classes=2)
print("Images:", data.shape, "| Labels:", labels.shape)

X_train, X_test, y_train, y_test = train_test_split(
    data, labels, test_size=0.20, random_state=42, stratify=labels)
del data, labels; gc.collect()
print("Train:", X_train.shape, "| Test:", X_test.shape)

```

2. Build the transfer-learning model

```

BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE

def make_ds(X, y, training=False):
    ds = tf.data.Dataset.from_tensor_slices((X, y))
    if training:
        ds = ds.shuffle(1024)
    ds = ds.batch(BATCH_SIZE)
    # cast uint8 -> float32 and apply MobileNetV2 preprocessing per batch only
    ds = ds.map(lambda x, y: (preprocess_input(tf.cast(x, tf.float32)), y),
                num_parallel_calls=AUTOTUNE)
    return ds.prefetch(AUTOTUNE)

train_ds = make_ds(X_train, y_train, training=True)
test_ds = make_ds(X_test, y_test) # not shuffled -> label order preserved

baseModel = MobileNetV2(weights="imagenet", include_top=False,
                        input_shape=(224, 224, 3))
baseModel.trainable = False # freeze pretrained backbone

headModel = baseModel.output
headModel = AveragePooling2D(pool_size=(7, 7))(headModel)
headModel = Flatten()(headModel)
headModel = Dense(128, activation="relu")(headModel)
headModel = Dropout(0.5)(headModel)
headModel = Dense(2, activation="softmax")(headModel)

model = Model(inputs=baseModel.input, outputs=headModel)
model.summary()

```

3. Compile and train

```

INIT_LR = 1e-4
EPOCHS = 10

model.compile(
    loss="categorical_crossentropy",
    optimizer=tf.keras.optimizers.Adam(learning_rate=INIT_LR),
    metrics=["accuracy"])

history = model.fit(train_ds, validation_data=test_ds, epochs=EPOCHS)

```

4. Evaluate and save

```
pred_probs = model.predict(test_ds)
preds      = np.argmax(pred_probs, axis=1)
true_labels = np.argmax(y_test, axis=1)

print("Accuracy :", accuracy_score(true_labels, preds))
print("Precision:", precision_score(true_labels, preds))
print("Recall   :", recall_score(true_labels, preds))
print(classification_report(true_labels, preds,
                           target_names=["Without Mask", "With Mask"]))

model.save("face_mask_detector.h5")
print("Saved to face_mask_detector.h5")
```

5. Face detection + mask classification (inference)

```
# OpenCV DNN face detector (Caffe) + our MobileNetV2 mask classifier
!wget -q ../deploy.prototxt -O deploy.prototxt
!wget -q ../res10_300x300_ssd_iter_140000.caffemodel -O res10.caffemodel

faceNet = cv2.dnn.readNet("res10.caffemodel", "deploy.prototxt")
maskNet = tf.keras.models.load_model("face_mask_detector.h5")

def detect_and_predict(frame, conf_thresh=0.5):
    (h, w) = frame.shape[:2]
    blob = cv2.dnn.blobFromImage(frame, 1.0, (300, 300),
                                  (104.0, 177.0, 123.0))

    faceNet.setInput(blob)
    detections = faceNet.forward()
    for i in range(detections.shape[2]):
        confidence = detections[0, 0, i, 2]
        if confidence < conf_thresh:
            continue
        box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
        (startX, startY, endX, endY) = box.astype("int")
        startX, startY = max(0, startX), max(0, startY)
        endX, endY     = min(w - 1, endX), min(h - 1, endY)
        face = frame[startY:endY, startX:endX]
        if face.size == 0:
            continue
        face = cv2.cvtColor(face, cv2.COLOR_BGR2RGB)
        face = cv2.resize(face, (224, 224))
        face = preprocess_input(face.astype("float32"))
        face = np.expand_dims(face, axis=0)
        (withoutMask, mask) = maskNet.predict(face, verbose=0)[0]
        label = "Mask" if mask > withoutMask else "No Mask"
        color = (0, 255, 0) if label == "Mask" else (0, 0, 255)
        label = f"{label}: {max(mask, withoutMask) * 100:.1f}%"
        cv2.putText(frame, label, (startX, startY - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
        cv2.rectangle(frame, (startX, startY), (endX, endY), color, 2)
    return frame
```

Output

The dataset yielded 4,072 cropped face images (3,257 train / 815 test). The model summary shows about 2.42M total parameters, of which only 164,226 are trainable (the new head); the rest belong to the frozen backbone.

```
Images: (4072, 224, 224, 3) | Labels: (4072, 2)
Train: (3257, 224, 224, 3) | Test: (815, 224, 224, 3)
Total params:      2,422,210 (9.24 MB)
Trainable params:   164,226 (641.51 KB)
Non-trainable params: 2,257,984 (8.61 MB)
```

Training over 10 epochs showed steady improvement, with validation accuracy rising from 0.88 to 0.93.

```
Epoch 1/10 accuracy: 0.7753 - loss: 0.5143 - val_accuracy: 0.8847 - val_loss: 0.2965
Epoch 5/10 accuracy: 0.9131 - loss: 0.2183 - val_accuracy: 0.9178 - val_loss: 0.2061
Epoch 10/10 accuracy: 0.9321 - loss: 0.1715 - val_accuracy: 0.9301 - val_loss: 0.1783
```

Evaluation on the held-out test set:

Metric	Value
Accuracy	0.9301
Precision	0.9377
Recall	0.9768

Per-class classification report (support in parentheses):

	precision	recall	f1-score	support
Without Mask	0.89	0.75	0.82	168
With Mask	0.94	0.98	0.96	647
accuracy			0.93	815
macro avg	0.92	0.86	0.89	815
weighted avg	0.93	0.93	0.93	815

Real-time inference on a webcam capture. The face detector localised the face and the classifier labelled it, drawing a red box for No Mask with the confidence score.

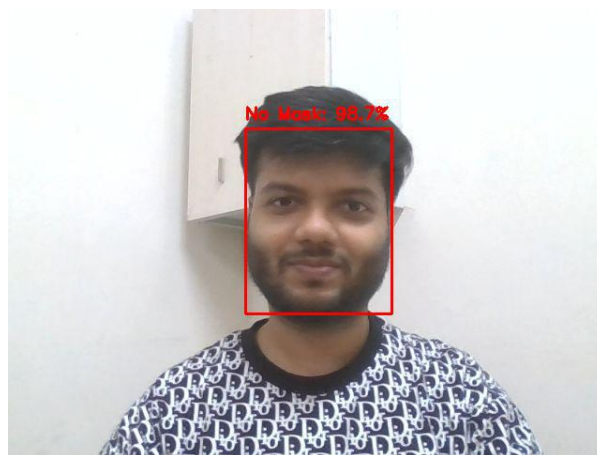


Figure 2.1 — Real-time inference: face detected and classified as “No Mask”.

Result

A real-time face mask detection system was built successfully using transfer learning. The frozen MobileNetV2 backbone with a small trained head reached 93.0% test accuracy, 93.8% precision and 97.7% recall, and correctly produced bounding boxes with live Mask / No Mask labels on webcam input — meeting all stated objectives.

Experiment 3: Build a CNN (Conv → ReLU → MaxPool) and Visualize Feature Maps

Aim: To build a Convolutional Neural Network using the Conv → ReLU → MaxPool pipeline, train it on MNIST, and extract and visualize the feature maps produced at each stage.

Framework: PyTorch.

Dataset: MNIST — 70,000 grayscale 28x28 images of handwritten digits 0–9 (60,000 train / 10,000 test).

Theory

A Convolutional Neural Network (CNN) learns visual features directly from pixels through three repeating operations. The convolution layer slides small learnable filters (kernels) across the image; each filter responds to a particular local pattern (an edge, a curve, a stroke) and produces a feature map showing where that pattern occurs. Stacking many filters yields many feature maps, one per channel.

The ReLU activation, $f(x) = \max(0, x)$, is applied element-wise after convolution. It introduces non-linearity (so the network can model complex functions) and clips all negative responses to zero, keeping only the locations where a feature was positively detected. In the visualisations this is why dark/negative regions disappear after ReLU.

The max-pooling layer downsamples each feature map by taking the maximum value in small (here 2x2) windows. This shrinks the spatial size (28→14→7), reduces computation, and gives a degree of translation invariance — the exact pixel position of a feature matters less. After two Conv→ReLU→Pool stages the maps are flattened and passed to a fully-connected layer that outputs the 10 digit-class scores.

Visualising feature maps makes the network interpretable: it shows what each filter has learned to react to and how the representation becomes more abstract and compact as data flows deeper.

Network Architecture

Layer	Input	Output
Conv2d(1→8, 3x3, pad 1)	1→28→28	8→28→28
ReLU + MaxPool(2x2)	8→28→28	8→14→14
Conv2d(8→16, 3x3, pad 1)	8→14→14	16→14→14
ReLU + MaxPool(2x2)	16→14→14	16→7→7
Flatten	16→7→7	784
Linear(784→10)	784	10 (class scores)

Procedure

- Load the MNIST IDX files, normalise pixel values to [0, 1], and add a channel dimension to get (N, 1, 28, 28) tensors.
- Wrap the data in TensorDatasets and DataLoaders for batching.
- Define SmallCNN: Conv(1→8) → ReLU → MaxPool, Conv(8→16) → ReLU → MaxPool, flatten, then a linear layer to 10 classes.
- Train for 2 epochs with the Adam optimiser and cross-entropy loss, then measure test accuracy.
- Pass a single test image through conv1, ReLU and pool individually, capturing the intermediate tensors.
- Plot each of the 8 channels at every stage to visualise the feature maps.

Source Code

1. Load MNIST from IDX files

```
import os, struct
import numpy as np

MNIST_DIR = '/content/drive/MyDrive/Datasets/MNIST'

def read_idx(path):
    """Parse an IDX (ubyte) file into a numpy array."""
    with open(path, 'rb') as f:
        _, _, ndims = struct.unpack('>HBB', f.read(4)) # 2 zero bytes, dtype, ndims
        shape = tuple(struct.unpack('>I', f.read(4))[0] for _ in range(ndims))
        data = np.frombuffer(f.read(), dtype=np.uint8)
        return data.reshape(shape)

X_train = read_idx(files['train_images']).astype(np.float32) / 255.0
y_train = read_idx(files['train_labels']).astype(np.int64)
X_test = read_idx(files['test_images']).astype(np.float32) / 255.0
y_test = read_idx(files['test_labels']).astype(np.int64)
print("Shapes:", X_train.shape, y_train.shape, X_test.shape, y_test.shape)
```

2. Define the CNN

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# add channel dim: (N, 28, 28) -> (N, 1, 28, 28)
train_ds = TensorDataset(torch.from_numpy(X_train).unsqueeze(1),
                          torch.from_numpy(y_train))
test_ds = TensorDataset(torch.from_numpy(X_test).unsqueeze(1),
                        torch.from_numpy(y_test))
train_loader = DataLoader(train_ds, batch_size=128, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=256)

class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, kernel_size=3, padding=1) # 28x28 -> 28x28
        self.conv2 = nn.Conv2d(8, 16, kernel_size=3, padding=1) # 14x14 -> 14x14
        self.pool = nn.MaxPool2d(2, 2)
        self.fc = nn.Linear(16 * 7 * 7, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # Conv -> ReLU -> MaxPool (28 -> 14)
        x = self.pool(F.relu(self.conv2(x))) # Conv -> ReLU -> MaxPool (14 -> 7)
        x = x.flatten(1)
        return self.fc(x)

model = SmallCNN().to(device)
print(model)
```

3. Train and evaluate

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()
EPOCHS = 2

for epoch in range(EPOCHS):
    model.train()
    running = 0.0
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        loss = criterion(model(xb), yb)
        loss.backward()
        optimizer.step()
        running += loss.item() * xb.size(0)
    print(f"Epoch {epoch+1}: train loss = {running/len(train_ds):.4f}")

# quick test accuracy
model.eval()
correct = 0
with torch.no_grad():
    for xb, yb in test_loader:
        xb, yb = xb.to(device), yb.to(device)
        correct += (model(xb).argmax(1) == yb).sum().item()
print(f"Test accuracy: {100*correct/len(test_ds):.2f}%")
```

4. Extract and visualize feature maps

```
import matplotlib.pyplot as plt

# pick one test image
img = torch.from_numpy(X_test[0]).unsqueeze(0).unsqueeze(0).to(device) # (1,1,28,28)
label = y_test[0]

model.eval()
with torch.no_grad():
    fmap_conv = model.conv1(img) # after Conv
    fmap_relu = F.relu(fmap_conv) # after ReLU
    fmap_pool = model.pool(fmap_relu) # after MaxPool

print("conv1:", tuple(fmap_conv.shape),
      "| relu:", tuple(fmap_relu.shape),
      "| pool:", tuple(fmap_pool.shape))

def show_maps(tensor, title):
    maps = tensor.squeeze(0).cpu().numpy() # (C, H, W)
    n = maps.shape[0]
    fig, axes = plt.subplots(1, n, figsize=(2*n, 2.4))
    fig.suptitle(title, fontsize=13)
    for i, ax in enumerate(np.atleast_1d(axes)):
        ax.imshow(maps[i], cmap='viridis')
        ax.axis('off'); ax.set_title(f'ch {i}', fontsize=8)
    plt.tight_layout(); plt.show()

show_maps(fmap_conv, "After Conv (8 channels)")
show_maps(fmap_relu, "After ReLU (negatives clipped to 0)")
show_maps(fmap_pool, "After MaxPool (downsampled 28->14)")
```

Output

Training and test results after 2 epochs:

```
Device: cpu
Epoch 1: train loss = 0.4784
Epoch 2: train loss = 0.1233
Test accuracy: 97.45%

Input label: 7
conv1: (1, 8, 28, 28) | relu: (1, 8, 28, 28) | pool: (1, 8, 14, 14)
The input test image — a handwritten digit 7:
```

Input (label 7)

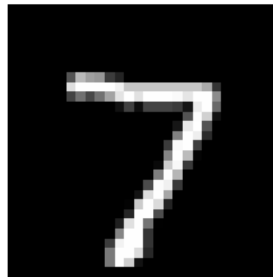


Figure 3.1 — Input image (label 7).

After the convolution layer, each of the 8 filters produces a different feature map. Some highlight the horizontal top stroke, others the diagonal, and channel 4 responds with a near-uniform high value (its negatives are still present here).

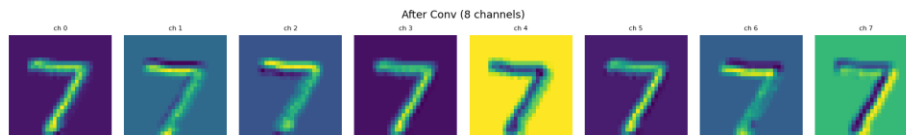


Figure 3.2 — After Conv (8 channels).

After ReLU, all negative responses are clipped to zero. Channel 4 collapses to near-empty, and the remaining maps keep only the positively-detected strokes — a cleaner, sparser representation.

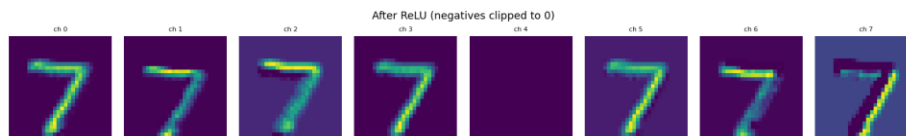


Figure 3.3 — After ReLU (negatives clipped to 0).

After max-pooling, every map is downsampled from 28x28 to 14x14. The digit's shape is preserved but coarser, reducing computation while retaining the salient features.

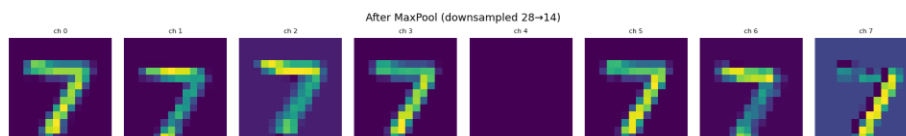


Figure 3.4 — After MaxPool (downsampled 28 → 14).

Result

A CNN following the Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool pipeline was built and trained on MNIST, reaching 97.45% test accuracy in just 2 epochs. The feature maps at each stage were extracted and visualised, clearly showing how convolution detects local patterns, ReLU sparsifies them by removing negatives, and pooling compresses the spatial resolution — fulfilling the objective of building and interpreting a CNN.